

System Design: Small Scale Search Engine

Crawler Crew

April 27, 2025



Group Members

- Jiongyi Shao: Computer Science, junior, EECS485/EECS482.
- Lian Duan: Computer Science, junior, EECS485/EECS445.
- Qianhui Zheng: Computer Science, junior, EECS482/EECS445.
- Ruikai Yang: Computer Science, junior, EECS445.
- Yuzhe Niu: Computer Engineering, junior, EECS485/EECS482.
- Zijian Su: Math, junior, EECS482 (quit later).

1 Project Summary

1.1 Team Roles and Responsibilities

At the beginning of the project, we discussed the strengths and expertise of each team member to ensure that everyone took on a role that aligned with their interests:

- **Lian Duan:** Focused on the reverse word index and indexing strategy, ensuring the proper management of data and efficient querying.
- **Jiongyi Shao:** Focused on the overall design of Index stream reader, including the handling of the index files and the integration with the ranker.
- **Yuzhe Niu:** Contributed to the crawler design, handling the redirection, and working on the connection between the ISR and the index files.
- **Ruikai Yang:** Contributed to the ranker design, working on the ranking algorithms and optimizing the page ranking process based on static and dynamic page attributes.
- **Qianhui Zheng:** Responsible for the overall architecture and design of the crawler, as well as handling the development of the user interface

1.2 Collaboration and Meetings

Throughout the project, we held weekly meetings to track our progress, discuss technical challenges, and coordinate tasks. These meetings were instrumental in making sure everyone was on the same page and working towards the same goal. Additionally, we used group chat and version control (Private Git: <https://github.com/zqhhhhh/CrawlerCrew.git>) to keep communication fluid and ensure that everyone had access to the latest codebase. Together we make many important decisions, which will be discussed detailedly in following sections.

2 Project Spreadsheet

The following table serves pretty much as a reference regarding who did what in LOC.

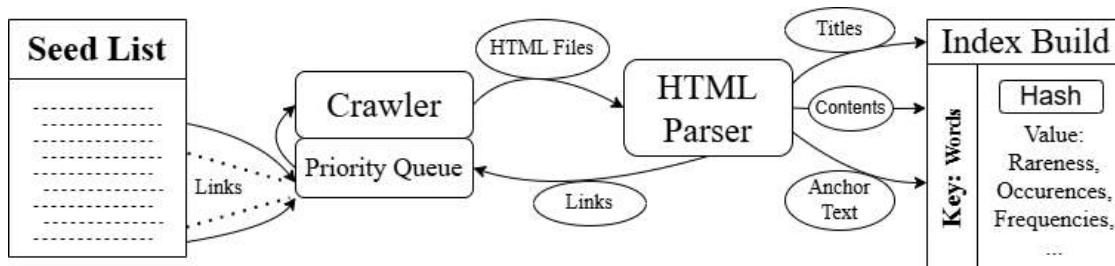
Components	Lian Duan	Jiongyi Shao	Yuzhe Niu	Ruikai Yang	Qianhui Zheng	Total
	LOC	LOC	LOC	LOC	LOC	LOC
HTML Parser	219	0	219	0	0	438
Crawler	813	813	813	813	813	4065
Index	386	0	0	0	385	771
Constraint solver	0	464	464	0	0	928
Query language	0	422	422	0	0	844
Ranker	161	0	0	642	0	803
Front end	0	0	0	0	589	589
Linux Server	0	0	0	541	0	541
Templates	943	603	603	526	738	3413
Distributed System	0	222	0	0	0	222
Subtotal	2522	2524	2521	2522	2525	12614

3 Crawler side Overview

In this section, we will show the whole process to get the inverted word index used in later sections. We will start from explaining how our crawler works, go deeply into how we parse the downloaded pages, and finally discuss the method we use to built an index with parsed results.

3.1 Overall Structure

The input of the crawler is a seed list filled with URLs, and the corresponding output is a set of documents, which are used to build the inverted word index. To achieve this, we need a front end to save all URLs visited to avoid double crawling. We also need to deal with broken HTMLs, handle redirects, and keep track of all global variables to recover from a crash.



From the crawler, we download HTML pages and pass them into the HTML parser. The links in the HTML pages are extracted, graded by their quality, and add back into the priority queue in the front end. All useful content will be classified as **Title** texts, **Body** texts, and **Anchor** texts. They will go though similar process and become a part of the inverted word index. The index will be saved as HashBlob, which will be detailedly discussed in later sections.

3.2 Crawler Strategy

We have come up with three kinds of crawling methods based on the catagory of web pages. The first method is to crawl all the pages in a website. This is specially designed for selected high quality domains, like wikipedia, nytimes, etc. We will restrict all links added to the frontier to be within that domain. That means higher quality, more relevant content, and less spam will be crawled. The pages crawled by this method will be builded to the first several chunks of index, which means we can give them a higher static score to shown our preference.

The second method is to crawl only home pages. That idea intuitively comes from the common difficulty to crawl down some big websites, like facebook, amazon, etc. Very often, what users need is the access to the homepage, and we can get considerable important information from a special seed list of about 100 home pages.

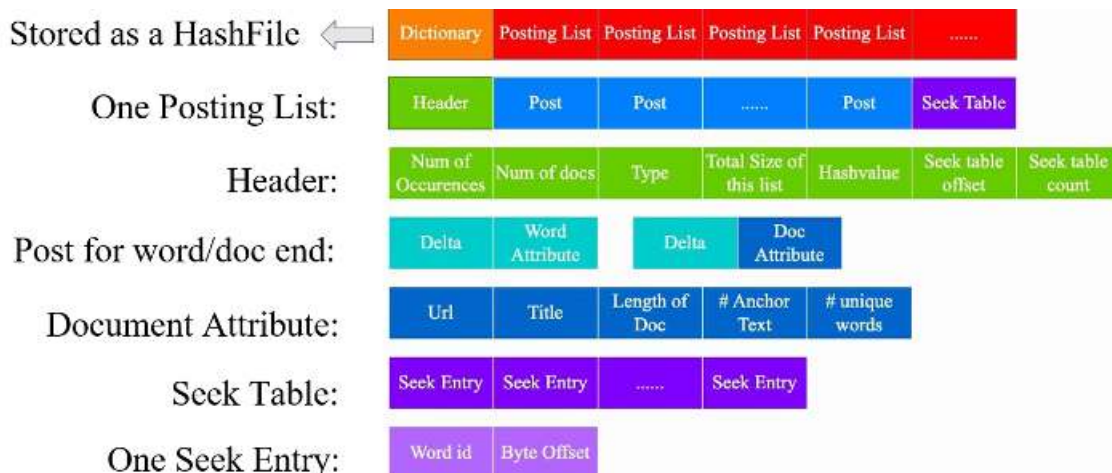
The third method is the default one, which is to crawl all the pages in a website, but with a limited depth. We set up a variable called **max_depth**, which requires the pages

crawled to be within a certain number of clicks from the seed page. Auto restarting and auto recovery are also implemented to ensure the crawler can continue to work even if it crashes. Global variables are stored in a working log file, which will be loaded when the crawler restarts. After crawling, the statistics of our results is shown in the table:

Statistic	Value
Number of Documents	30M
Number of Unique Words	229000
Number of Bytes of Source Material	500 GB
Number of Bytes in Filesystem Footprint	600 GB
Crawl Speed (per machine)	1500 pages/min
Query Serve Time	1500 ms

3.3 Inverted Word Index

Inverted word index is a very critical part of a search engine. It is used to store the mapping between words and the documents they appear in. In later experiments, we found that the response time of the search engine is highly related to the size of the index. To reduce the size of the index, the first thing we do is to kick out stopwords. Words like “and”, “or”, “the” are appearing almost anywhere in the web pages, and they are not very useful for searching. After filtering out **570** stop words, we cut down the size of the index by **20%**. A view of the structure for the index is as follows:



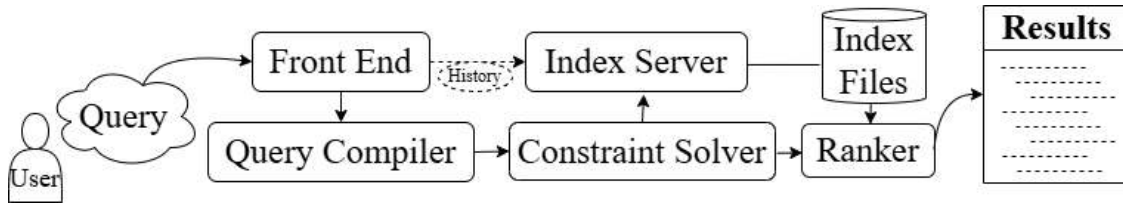
Another very important thing is to use variable length encoding. We save the positions of occurrences as delta from the last position, which can greatly reduce the size of the index. However, that also introduces a problem: the deltas can be largely different from each others. For example, if we consider the occurrence of a rare word ”pipefish”, it might appear multiple times in a single document, but each document that has the word might be very far away from the others. That means we could save a lot of space by assigning a small space to store small deltas, and assigning larger space for larger deltas. After applying variable length encoding, the size of the index is reduced by another **50%**.

4 Query Side Overview

In this section, we will go deeply into the query side of the project, including the query parser and the constraint solver. We will start from the overall structure of Query serve side, go deeply into the expected structure of a query, and finally discuss the implementation of the query parser and the constraint solver.

4.1 Overall Structure

The input of the query server is a query string typed in by the user. The corresponding output should be at least a list of URLs, sorted by the relevance to the query. To achieve this, we need a front end to receive the input string and present the final results. The query compiler will parse the input string according to query grammar and generate a query tree. The constraint solver will generate a similar ISR tree from the query tree, which will be passed into the Index server. Finally, we request the inverted word index from the Index server and use the ranker to get the final results. The overall structure is shown by the following diagram:



4.2 Expected Structure of a Query

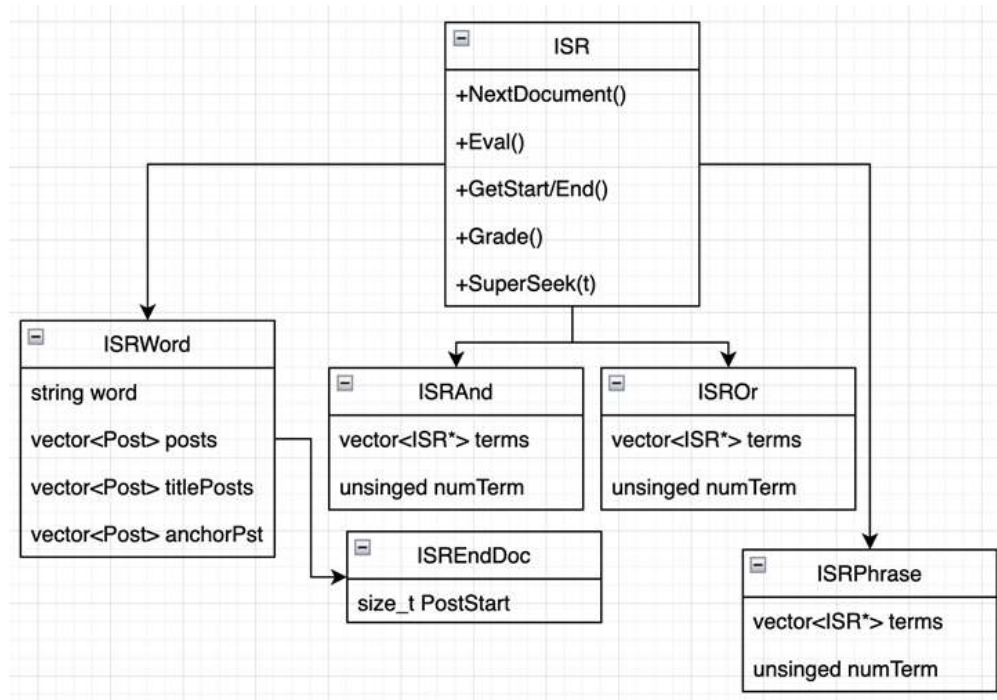
We support three kinds of logical operators (AND, OR, and PHRASE) and two kinds of decorators (#TITLE and \$Anchor). The expected structure of a query is shown by the following diagram: The logic is based on a simple assumption: there are only single words,

```
<Query> ::= <And> { '|' <And> }
<And>    ::= <Part> { <Part> }
<Part>   ::= '(' <Query> ')' | <Phrase> | ['$ ', '#']word
<Phrase> ::= " word { word } "
```

no other logical operators inside a phrase query. That means, the query "quick fox" — "brown fox" is valid, but "quick — brown fox" is not. That explains why a phrase, single word (possibly with decorators), or a subquery inside parenthesis is defined as the leaf node of the query tree. Here we call them as parts. Different parts connected with white space are considered as AND queries, and different AND queries connected with '—' are considered as OR queries.

4.3 ISR Structure

Following the structure of the query tree, we have set up different kinds of nodes in the ISR tree. We use the following diagram to show the structure of the ISR related classes:



Everything starts from the base class ‘ISR’. ‘ISR’ is an abstract class, and it has four subclasses: ‘ISRWord’, ‘ISROr’, ‘ISRAnd’, and ‘ISRPhrase’. ‘ISREnddoc’ is further derived from ‘ISRWord’, since it has some more information to be stored. To be noted, every real ISR will include a vector that stores the occurrences within the current document. All these ISRs have common member functions that can be called.

- ‘NextDocument()’: updates the page information to the next page that matches this ISR.
- ‘Eval()’: updates the vector of posts stored in this ISR according to new page information.
- ‘GetStart/End()’: return the starting and ending location of the current document.
- ‘Grade()’: return the score of this page corresponding to the current page information.
- ‘SuperSeek(t)’: return the first page that matches current ISR after position ‘t’.

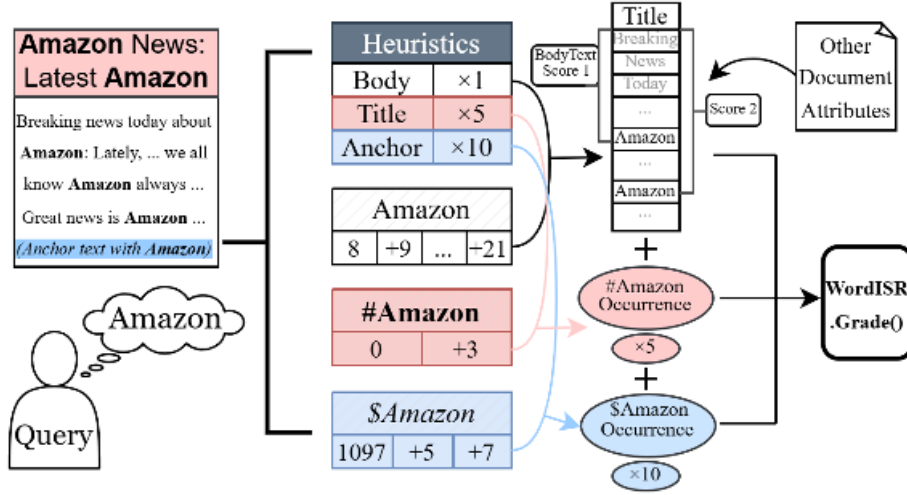
With these common member functions, it becomes much easier for ranker to rank the pages. After initializing the query ISR, we can call ‘NextDocument()’ and ‘Eval()’ to get the page information, and then call ‘Grade()’ to get the score of each page. A priority queue can be maintained to store the Top 10 results. We will discuss the ranking strategy detailedly in the next section.

5 Ranking Strategy

Ranking is one of the most important parts of a search engine. Since the process takes a lot of factors into consideration, it is usually a complex process. Luckily, we have a great ISR system that provides us with vectors of locations for occurrences within a certain page and all other document attributes. What we discuss here is given these information, how we grade a simple Word ISR, an And / Or ISR, and how we grade a phrase ISR.

5.1 Word ISR Grading

The following diagram shows our grading strategy for a Word ISR:



The grading strategy is based on the following assumptions:

- The more occurrences of a word in a page, the more relevant it is.
- The closer the occurrences are to the title, the more user want this page.
- An occurrence in title / anchor text is more important than an occurrence in the body.

Given a word ISR, we can access three vectors: 'title', 'anchor', and 'body'. We will count the number of occurrences in 'title' and 'anchor', assign different weights to them, and then add them up as 'special score'. We then iterate through the body text vector, calculate its relative distance to the title, and add up the scores of all occurrences as 'normal score'. Finally we will calculate a satisfying score by considering the total length of the page and the total occurrence of the specific word in all chunks. The sigmoid function is used to map the score to a value between 0 and 1:

$$\text{standardized_grade} = \frac{1.0}{1.0 + \exp\left(-\text{slope} \times \left(\frac{\text{grade}}{\text{satisfying_grade}} - 1\right)\right)} \quad (1)$$

, where 'slope' is a parameter that can be adjusted to control the sharpness of the curve. The reason we restrict the score to be between 0 and 1 is to make it a fair comparison with other ISRs. We will discuss this more detailedly in the next section.

5.2 And / Or ISR Grading

Our grading strategy is based on the following assumptions:

- For a ‘A | B’ search, a satisfying page for ‘A’ should have a similar score to a satisfying page for ‘B’.
- For a ‘A B’ search, a satisfying page should at least have both the occurrences of ‘A’ and ‘B’.

Based on the first assumption, we come up with following grading algorithms for Or ISR:

Algorithm 1: OrISR Grading Algorithm

```
Input: terms: list of search terms
Output: standardized_grade
standardized_grade  $\leftarrow$  0
foreach  $term \in terms$  do
    term_grade  $\leftarrow$  term.Grade()
    if  $term\_grade > standardized\_grade$  then
        | standardized_grade  $\leftarrow$  term_grade
    end
end
return standardized_grade
```

Since we’re simply taking the maximum score among all terms, we need to introduce standardization to make sure the comparisons are fair. Else, if we have the score of a matching page for And ISR significantly higher than a single Word ISR, we will only get matching results for the And query in final results. That’s not what we want, so we include an average pooling in the grading algorithm for And ISR:

Algorithm 2: AndISR Grading Algorithm

```
Input: terms: list of search terms
Output: standardized_grade
standardized_grade  $\leftarrow$  0
foreach  $term \in terms$  do
    term_grade  $\leftarrow$  term.Grade()
    standardized_grade  $\leftarrow$  standardized_grade + term_grade
end
standardized_grade  $\leftarrow$  standardized_grade / |terms|           // Average pooling
return standardized_grade
```

From the Word ISR grading, we can observe that when the grade achieves the satisfying threshold, its corresponding standardized score will be 0.5. By average pooling, we can control the satisfying threshold for And / Or ISR to be 0.5 as well. That solves the problem of unfairness when comparing the scores of different different kinds of ISRs inside an Or ISR. With these algorithms, what is left is the grading algorithm for Phrase ISR, which is pretty much the same. For space limit, we will not discuss it here.

6 Search Engine Functionality

This section summarizes the key functionalities of Crawler Crew, highlighting the engine’s capabilities across different levels of operation.

6.1 Crawler & Parser

Crawler Crew efficiently parses HTML documents to extract words and links, which are essential for indexing. It can crawl up to 30 million pages using 12 Google Cloud virtual machines and 3 gaming laptops, supporting both HTTP and HTTPS protocols. The crawler manages a frontier of URLs, prioritizing them to ensure a well-organized crawl. It respects robots.txt and supports multithreading for parallel processing. The crawler is also restartable and has automatic crash recovery to ensure robustness.

6.2 Reverse Word Index

Crawler Crew indexes 30 million documents, retaining parsed HTML content. The index is divided into 90 MB chunks, storing metadata such as URL, title, body, and anchor text. Delta encoding and seek tables improve performance, ensuring efficient access to the index. This setup enables quick and scalable querying for large datasets.

6.3 User Interface

Crawler Crew offers a simple command-line interface (CLI) and an HTTP server as a wrapper UI, allowing users to perform searches and receive results with clickable URLs and titles. This basic yet functional interface provides an intuitive way to interact with the engine and access search results.

6.4 Query Parsing, Ranker, and Constraints

The query parser supports logical operators (AND, OR, phrases) and compiles queries into intermediate search results (ISRs). The ranker evaluates static page attributes like URL length, domain, and anchor texts to prioritize results. Matches in titles and anchor texts are given higher weight. Heuristic methods further refine the ranking process, emphasizing the context of word positions, particularly in titles and anchor texts.

6.5 Distributed Architecture and Advanced Features

Crawler Crew operates on a distributed system, with crawling, indexing, and query processing handled by 12 Google Cloud virtual machines and 3 gaming laptops. This architecture ensures efficient performance even under high traffic. Additionally, the ranker is tuned using a training set, and features like stemming, stop words, and automatic recrawl are incorporated to enhance result accuracy and freshness.

7 Distributed Architecture and Communication

7.1 Crawler and Index Builder

Crawler Crew’s crawling, indexing, and querying processes are executed across multiple machines in a distributed setup, utilizing Google Cloud virtual machines and personal gaming laptops. The crawler runs in parallel on 15 machines, each responsible for retrieving and processing a portion of the web. The crawling task is distributed, with each machine managing a subset of the URLs. After crawling, the documents are parsed and indexed locally. Each machine stores its portion of the index in chunks, reducing the load on any single machine.

7.2 Query Processing and Communication

When a request comes in, each machine compares it against the index on the machine and returns the top 10 most-matching URLs. The results are then forwarded on to a central server where they’re aggregated, ranked, and duplicate results eliminated. This way, the most relevant and unique results are presented back to the user.

Machine-to-machine communication is facilitated by messaging protocols and shared resources. The individual machines ask for results from the local index and then forward these results to the central node for aggregation. The distributed communication model ensures scalability and efficiency by dealing with a large volume of queries as well as datasets.

8 Filesystem Footprint and Data Layout

Within our system, the HTML files are not stored on the local device as raw HTML when they are downloaded by the crawler. Rather, they are forwarded on immediately to the parser. The parsed output exists in a very structured format: the title words on the first line, the body text on the next line, and the anchor text on the third line. These parsed results are temporarily stored on a local basis as a means of backing up the data. After the parsing operation occurs, these local backup copies are sent into a HashTable function. The HashTable function creates a HashBlob. The created HashBlob gets stored within the root directory of the virtual machine. This method of operation permits us the ability to hold a reference to the parsed content while allowing us to keep the data processed and indexed efficiently.

The choice of keeping the parsed results as a local backup, rather than submitting them directly through the pipeline, exists mainly because of the necessity of keeping the index files’ continuous structure. By storing the anchor text as individual files and integrating these later on, we ensure the continuity and integrity of the index structure. The choice also provides room for rebuilding the index structure and changing the layout thereof, making sure that any subsequent changes or optimisations are easily integrated into the existing system. This method of keeping the files ensures a scalable and adaptable procedure of indexing while providing a good basis for efficient management of large-scale web information.

9 Known Bugs and Further Work

While our project has made significant progress, there are a few known bugs and areas for further development that need to be addressed in future versions. These issues primarily concern the crawling process, the handling of anchor text, and adherence to web scraping policies. Below are the main known bugs and suggestions for future work:

- **Lack of Bloom Filter for Visited URLs:** Currently, the crawler does not utilize a Bloom filter to efficiently track visited URLs. This limitation can result in redundant crawling of the same URLs, increasing processing time and resource usage. Implementing a Bloom filter would significantly speed up the crawling process by quickly identifying whether a URL has already been visited.
- **Missing Snapshot Feature:** At this point, the crawler does not implement a snapshot feature. A snapshot will enable users to see where exactly the query occurs in the returned pages. Further work will include enabling this feature, which is feasible since we have already stored all parsed documents for archive use.
- **Anchor Text Handling Causing Slowdowns:** Storing anchor text separately from the body and title text, while necessary for maintaining index structure integrity, has led to performance issues. Specifically, the crawler's speed decreases as more anchor text is added to the system. The process of storing, indexing, and later integrating this anchor text could be optimized to improve the crawler's efficiency.
- **Robots.txt Compliance:** The crawler does not fully adhere to the robots.txt file of websites, which is a potential issue. The current implementation does not always check or respect the crawling directives set by webmasters, which could lead to scraping of restricted content or violating web scraping policies.

In conclusion, while Crawler Crew has made great progress in crawling, indexing, and query processing, there are several areas for improvement. Key future work includes implementing a Bloom filter for visited URLs, adding a snapshot feature, optimizing anchor text handling, and ensuring full compliance with robots.txt. Addressing these issues will enhance the performance, scalability, and ethical compliance of the system, enabling it to handle larger datasets and provide more accurate and efficient search results.